LOOPHOLE LEAGUE

Making Responsibility Visible in Pipelines

There are different types of loopholes. Some break systems at scale. Some actually drive innovation.

The ones that matter are **hidden**, **unclear in perception**, and show impact **at scale**.

Made with GAMMA

YOUR INSIGHT

When Do Loopholes Really Matter?

Value is Hidden

The impact of a loophole isn't obvious — it's buried inside the system's behavior.

Perception is Unclear

Different people interpret the same gap differently — is it a bug or a feature?

Impact Shows at Scale

Small gaps become serious failures only when the system runs at real volume.

Not every loophole should be fixed — some add flexibility or value.

So the real question is:

Which loopholes do we actually need to fix?

Made with GAMMA

THE SOLUTION

That's Where Loophole League Comes In

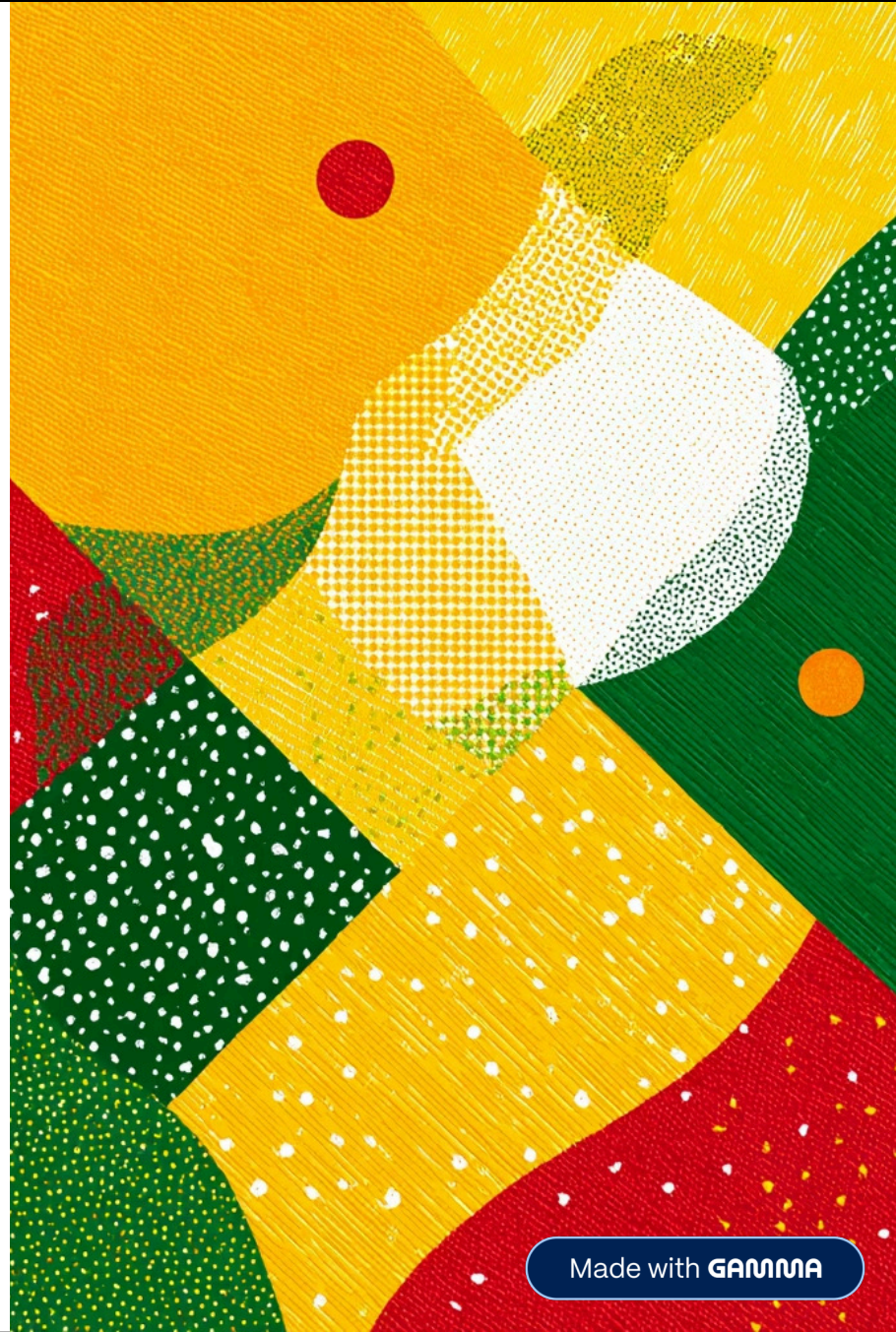
Goal: **Proper attribution of responsibility** in a multi-tool system.

When something goes wrong...

The problem is often *not* just the final output.

It can be earlier in the chain

Or something indirectly affecting the output from upstream.



Made with GAMMA

THE PROBLEM

Debugging the Wrong Thing

Function-Specific Tools

Most tools are siloed. Only the original developer really knows what a tool does — often gleaned from comments alone.

No Full Chain Visibility

When things break, we can't see the full pipeline. Dependencies are invisible. We're operating in the dark.

Wrong Part Gets Debugged

Without clear attribution, engineers chase the wrong component — wasting hours on symptoms, not causes.

⚠ I've personally spent hours debugging the wrong part just because I couldn't see how things were connected.

Made with GAMMA

WHAT YOU BUILT

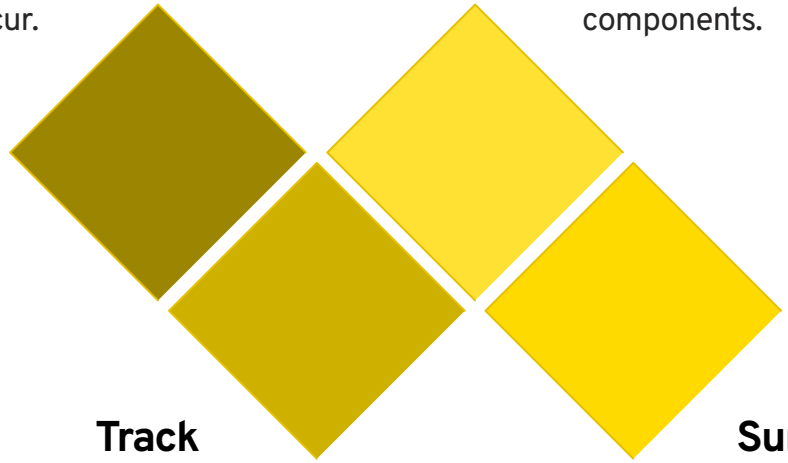
What Loophole League Does

Identify

Detect where and how failures occur.

Assign

Allocate points to contributing components.



Track

Monitor events across the entire pipeline.

Surface

Reveal root causes beyond the direct failure.

The direct failing component is **not always the biggest culprit** — something earlier in the chain may be the true source of responsibility.

Made with GAMMA

ATTRIBUTION SYSTEM

How Attribution Works

01

Build the Graph

Take the run and build a graph of all the steps. Each step is connected to what it depends on.

02

Start from Failure

Begin at the final output where the failure shows up and trace backwards through the system.

03

Push Blame Backwards

Each step shares responsibility with its dependencies — propagating accountability up the chain.

04

Group & Normalize

Group by tool and normalize so we can compare across the entire pipeline fairly.

05

Surface the Culprit

At the end, we know **which tool actually contributed the most** to the problem.

Made with GAMMA

TWO-LEVEL ATTRIBUTION

System Logs, Then Human Signal

Level 1 — System Logs

Automated Attribution

Attribution is first computed directly from system logs — objective, reproducible, and complete.

Level 2 — User Responses

Human Frustration as Data

Comments like *"this structure sucks"* or *"this doesn't match the document"* are used to adjust tool weights and guide attribution.

Human frustration becomes a signal, not a guess.

Developers are human. When things feel broken, they say so. Instead of ignoring that, Loophole League uses it to increase weight for specific tools — making attribution more aligned with real-world experience.

So we stop guessing where the problem is.

Made with GAMMA

AGENT FLOW

How the System Executes

1

Read Structure & Docs

Reads the pipeline structure and all supporting documents to understand the intended flow.

2

Validate & Regenerate

Checks if the structure makes sense. If not, generates a better one automatically.

3

Build Section by Section

Constructs the report using retrieval + generation across each pipeline segment.

At the same time, every tool call is logged, all dependencies are tracked, and everything is connected into the attribution graph.

Made with GAMMA

OUTPUT

What You See Instead of Just Logs

Clear Attribution

"Structure is 72% responsible → fix this" — no ambiguity about where to look.

Recommendations

Actionable next steps surfaced directly from the attribution analysis.

Pipeline Visual

A dependency graph of the entire run — see how components connect and where responsibility flows.

Expandable Logs

Raw log details available on demand — full transparency without overwhelming the default view.

The goal is to **show which part of the system actually matters** — not just dump raw data.

Made with GAMMA

WHY & CLOSING

Loopholes Aren't Just Errors

I built this because root cause analysis has always been frustrating. I've spent hours debugging the wrong part — and the way users react when things break tells you something real about where the system is failing.

They're Gaps in Responsibility

Loopholes aren't just bugs or edge cases — they're places where accountability goes unassigned.

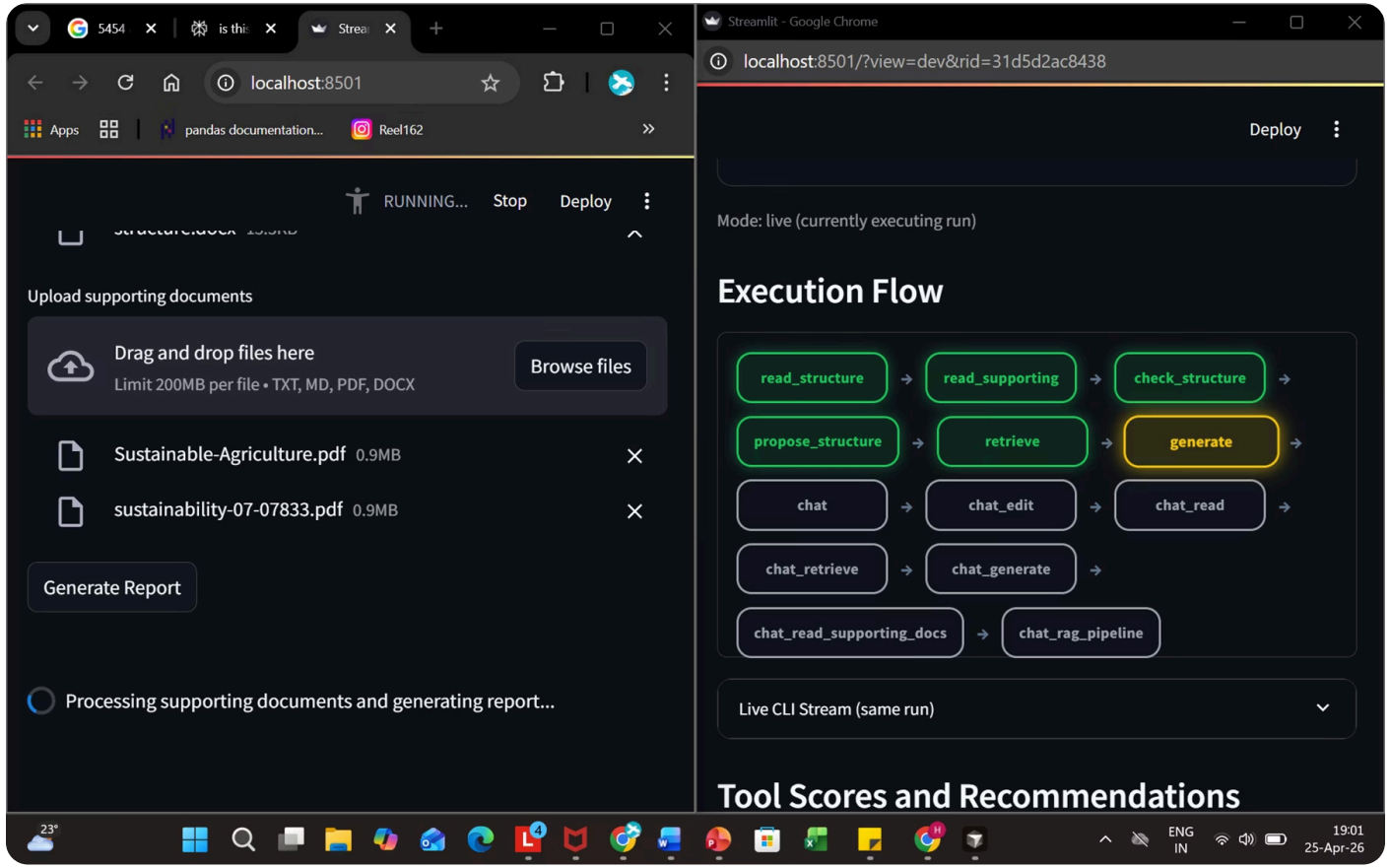
We Make Them Visible

Loophole League helps developers see what's actually going on and understand where responsibility really lies.

You Decide What to Fix

Once visible, you can choose — which loopholes to fix, and which ones to use for flexibility.

Made with GAMMA



Left: The Report generating Agent accepting input. Right: Developer view with Execution, flow, Tool Scores and recommendations, CL logs, Log History, Aggregate scores

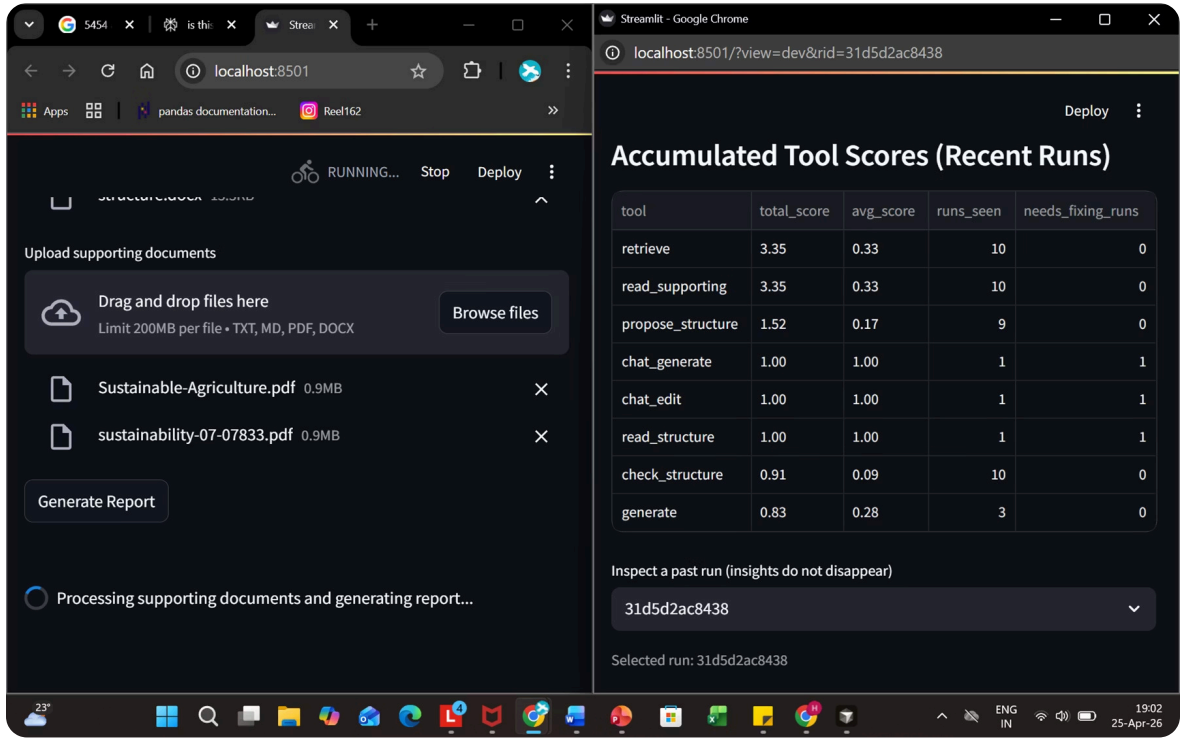
Made with GAMMA

Tool Scores and Recommendations

tool	score	needs_fixing	action
generate	0.27	NO	Monitor behavior
retrieve	0.27	NO	Check input dependencies
read_supporting	0.27	NO	Check input dependencies
propose_structure	0.13	NO	Optimize performance; Check input depenc
check_structure	0.07	NO	Monitor behavior

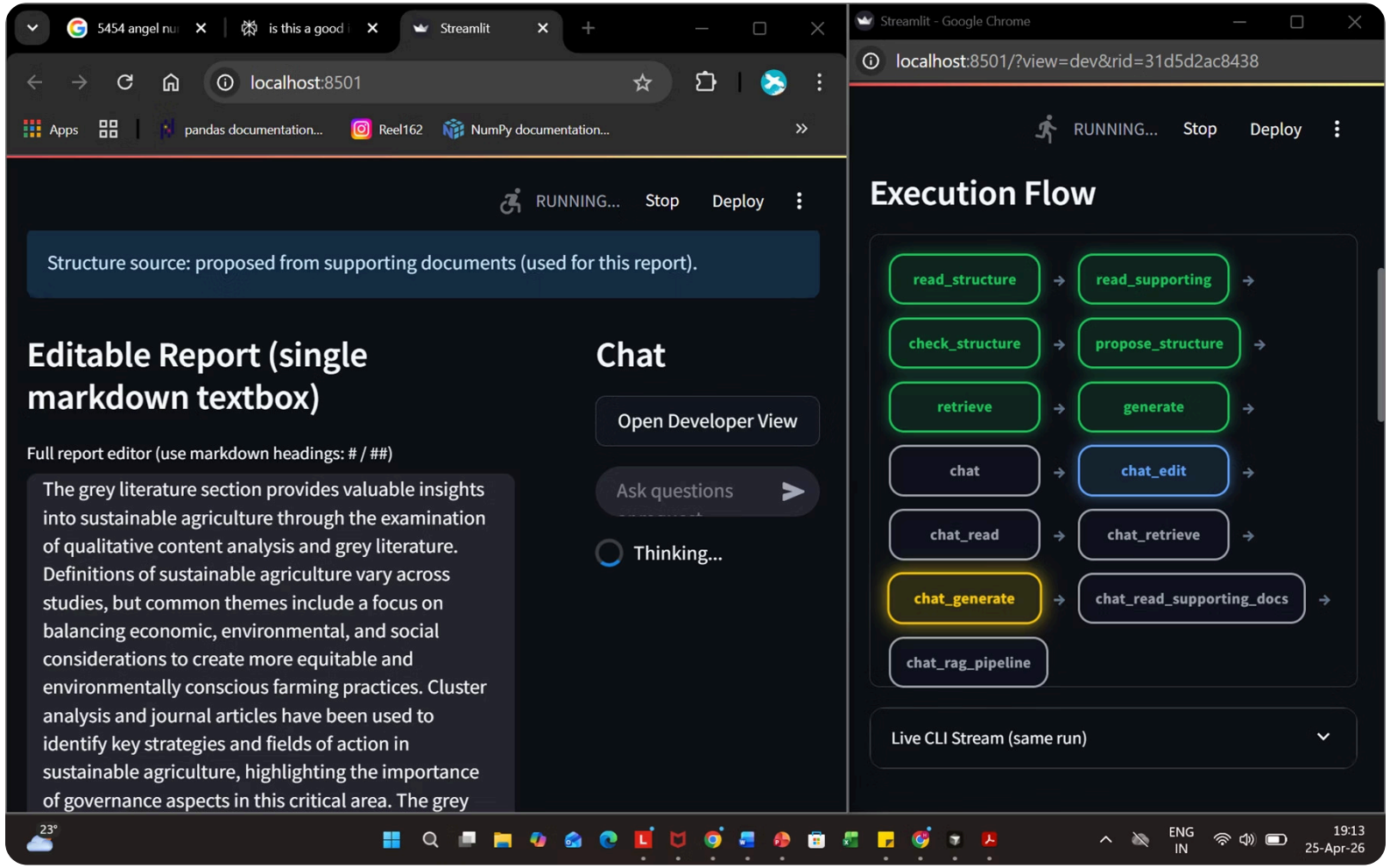
Example of tool scores in this case these functions are performing exactly as they should based on the "satisfaction" of its output fn.

Made with GAMMA



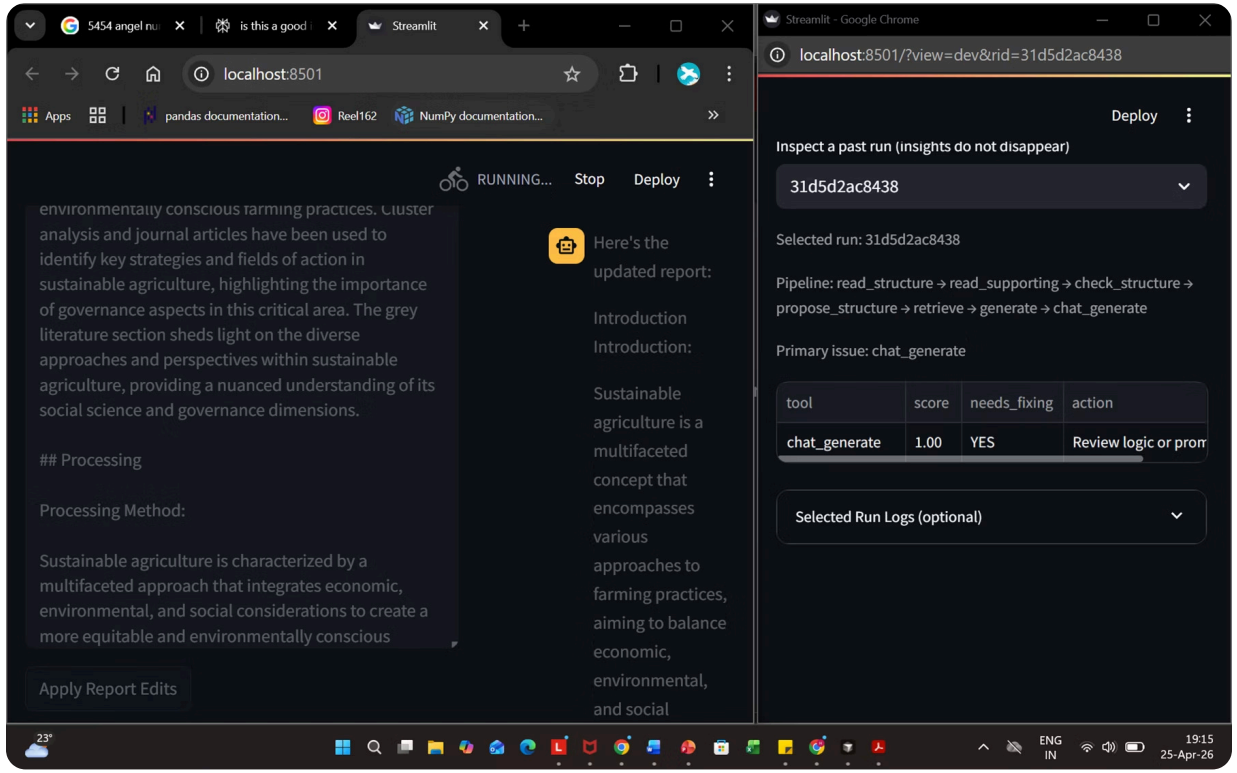
Accumulated scores and Run gistory with individual scores logged. [Chat is logged here as high becasue it isnt evaluated yet, hence it is important to keep an eye on the execution flow above]

Made with GAMMA



Chat Functionality

Made with GAMMA



Here, I told the chat that the section on "Grey Information" was irrelevant to me. And instead of going to structure component this system flagged the chat_generate tool because this is what verified relevance in this system. [Basically it showed that "relevance" was more important to me than "structure" and regenerated a new report without this section]

Made with GAMMA